Homework 1 — Question 3

This notebook implements the robust sparse-regression MILP from Question 2, Part B, and reports the requested results for Question 3 using `sublets.csv`.

Imports

In [1]:
```
using JuMP, Gurobi, CSV, DataFrames, LinearAlgebra, Statistics
```

Load and preprocess data

We center `rent` and standardize every feature column (no intercept).

In [2]:
```
df = CSV.read("sublets.csv", DataFrame)
feature_names = names(df)[2:end]   # all columns except rent

y = Vector{Float64}(df.rent)
y = y .- mean(y)

X = Matrix{Float64}(df[:, Not(:rent)])
X = (X .- mean(X, dims=1)) ./ std(X, dims=1)

n, p = size(X)
println("n = $n, p = $p")
println("features: ", feature_names)
```

```
n = 30, p = 7
features:
["sqft", "beds", "baths", "walk_t", "laundry", "parking", "floor"]
```

The MILP solver

Complete the function below. It should implement your model from Question 2 Part B.

Return a `NamedTuple` with fields `obj`, `u`, `w`, `β`, `s`.

In [3]:
```
function solve_robust_sparse(X, y, λ, ρ, k; M=1e3, solver_output=0)
    n, p = size(X)
    @assert length(y) == n

    model = Model(Gurobi.Optimizer)
    set_optimizer_attribute(model, "OutputFlag", solver_output)
```

```
    ################## ADD YOUR CODE AFTER THIS LINE ##################

    @variable(model, β[1:p])            # regression coefficients
    @variable(model, s[1:p], Bin)       # s[j] = 0 forces feature j to
    @variable(model, t[1:p] >= 0)       # t[j] ≥ |β[j]|
    @variable(model, u >= 0)            # u ≥ ‖y − Xβ‖∞
    @variable(model, w >= 0)            # w ≥ ‖β‖∞

    # u ≥ ‖y − Xβ‖∞ :  −u ≤ yᵢ − xᵢᵀβ ≤ u  for every listing i
    resid = y .- X * β
    @constraint(model, [i=1:n],  resid[i] <= u)
    @constraint(model, [i=1:n], -resid[i] <= u)

    # t[j] ≥ |β[j]|
    @constraint(model, [j=1:p],  β[j] <= t[j])
    @constraint(model, [j=1:p], -β[j] <= t[j])

    # w ≥ |β[j]| for every j, i.e. w ≥ ‖β‖∞
    @constraint(model, [j=1:p], t[j] <= w)

    # big-M link: s[j] = 0 ⟹ β[j] = 0
    @constraint(model, [j=1:p],  β[j] <= M * s[j])
    @constraint(model, [j=1:p], -β[j] <= M * s[j])

    # cardinality: ‖β‖₀ ≤ k
    @constraint(model, sum(s) <= k)

    @objective(model, Min, u + λ * sum(t) + ρ * w)

    optimize!(model)

    status = termination_status(model)
    status == MOI.OPTIMAL || error("Gurobi did not solve to optimality

    β_val   = value.(β)
    s_val   = value.(s)
    u_val   = value(u)
    w_val   = value(w)
    obj_val = objective_value(model)

    ################## ADD YOUR CODE BEFORE THIS LINE ##################

    return (obj=obj_val, u=u_val, w=w_val, β=β_val, s=s_val)
end
```

Out[3]: solve_robust_sparse (generic function with 1 method)

In [ ]:
```
"""Pretty-print a solution."""
function report_solution(sol; feature_names=feature_names, atol=1e-6)
    println("objective = $(round(sol.obj, digits=4))")
    println("u         = $(round(sol.u, digits=4))")
```

```
    println("w         = $(round(sol.w, digits=4))")
    selected = findall(j -> sol.s[j] > 0.5 && abs(sol.β[j]) > atol, 1:
    println("selected features:")
    for j in selected
        println("  $(feature_names[j]): β = $(round(sol.β[j], digits=4
    end
    if isempty(selected)
        println("  (none)")
    end
    return selected
end
```

Out[ ]: report_solution

Part (a)

For $\lambda = 0.25$, $\rho = 0.5$, and $k = 3$, the optimal objective value is **404.5056**, with optimal residual level

$$u = \|y - X\beta\|_\infty = \mathbf{196.8252}.$$

The selected features and coefficients are:

- `sqft` : $\beta_{\text{sqft}} = 137.2516$
- `beds` : $\beta_{\text{beds}} = 188.6077$
- `laundry` : $\beta_{\text{laundry}} = 127.6468$

The cell below generates these results.

In [5]:
```
sol_a = solve_robust_sparse(X, y, 0.25, 0.5, 3)
report_solution(sol_a);
```

```
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
objective = 404.5056
u         = 196.8252
w         = 188.6077
selected features:
  sqft: β = 137.2516
  beds: β = 188.6077
  laundry: β = 127.6468
```

Part (b)

The cell below compares the three uncertainty regimes with $k = 3$.

In [6]:
```
cases_b = [
```

```
    ("A only (λ=0.25, ρ=0)", 0.25, 0.0),
    ("B only (λ=0, ρ=1)",    0.0,  1.0),
    ("both   (λ=0.25, ρ=0.5)", 0.25, 0.5),
]

for (label, λ, ρ) in cases_b
    println("="^60)
    println(label)
    sol = solve_robust_sparse(X, y, λ, ρ, 3)
    report_solution(sol)
    println()
end
```

```
============================================================
A only (λ=0.25, ρ=0)
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
objective = 306.1088
u         = 191.6652
w         = 216.0
selected features:
  sqft: β = 120.4273
  beds: β = 215.1954
  laundry: β = 122.1519

============================================================
B only (λ=0, ρ=1)
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
objective = 376.0865
u         = 230.2808
w         = 145.8057
selected features:
  sqft: β = 137.2516
  beds: β = 145.8057
  laundry: β = 145.8057

============================================================
both   (λ=0.25, ρ=0.5)
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
objective = 404.5056
u         = 196.8252
w         = 188.6077
selected features:
  sqft: β = 137.2516
  beds: β = 188.6077
  laundry: β = 127.6468
```

Answer to (b)

The selected features and coefficients under each uncertainty regime are:

$\mathcal{A}$ only: $(\lambda, \rho) = (0.25, 0)$

- `sqft` : $\beta_{\text{sqft}} = 120.4273$
- `beds` : $\beta_{\text{beds}} = 215.1954$
- `laundry` : $\beta_{\text{laundry}} = 122.1519$

$\mathcal{B}$ only: $(\lambda, \rho) = (0, 1)$

- `sqft` : $\beta_{\text{sqft}} = 137.2516$
- `beds` : $\beta_{\text{beds}} = 145.8057$
- `laundry` : $\beta_{\text{laundry}} = 145.8057$

Both: $(\lambda, \rho) = (0.25, 0.5)$

- `sqft` : $\beta_{\text{sqft}} = 137.2516$
- `beds` : $\beta_{\text{beds}} = 188.6077$
- `laundry` : $\beta_{\text{laundry}} = 127.6468$

The ℓ_1 penalty from $\mathcal{A}$ charges every unit of total coefficient magnitude, so it shrinks the coefficients overall but does not directly prevent one coefficient from being much larger than the others. The ℓ_∞ penalty from $\mathcal{B}$ charges only the largest magnitude, encouraging the largest coefficients to level off.

Part (c)

The cell below compares the sparse model ($k = 3$) with the dense model ($k = 7$) at $\lambda = 0.25$ and $\rho = 0.5$.

In [7]:
```
println("===== k = 3 (sparse) =====")
sol_sparse = solve_robust_sparse(X, y, 0.25, 0.5, 3)
report_solution(sol_sparse)
println()

println("===== k = 7 (dense) =====")
sol_dense = solve_robust_sparse(X, y, 0.25, 0.5, 7)
report_solution(sol_dense);
```

```
===== k = 3 (sparse) =====
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
objective = 404.5056
u         = 196.8252
w         = 188.6077
selected features:
  sqft: β = 137.2516
  beds: β = 188.6077
  laundry: β = 127.6468

===== k = 7 (dense) =====
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
objective = 372.8233
u         = 126.5214
w         = 196.3027
selected features:
  sqft: β = 155.9765
  beds: β = 196.3027
  walk_t: β = -60.5236
  laundry: β = 113.3056
  parking: β = 15.6863
  floor: β = -50.8071
```

Answer to (c)

The optimal residual levels are $u = 196.8252$ for the sparse model ($k = 3$) and $u = 126.5214$ for the dense model ($k = 7$).

The nonzero coefficients for $k = 7$ are:

- `sqft` : $\beta_{\text{sqft}} = 155.9765$
- `beds` : $\beta_{\text{beds}} = 196.3027$
- `walk_t` : $\beta_{\text{walk_t}} = -60.5236$
- `laundry` : $\beta_{\text{laundry}} = 113.3056$
- `parking` : $\beta_{\text{parking}} = 15.6863$
- `floor` : $\beta_{\text{floor}} = -50.8071$

By insisting on at most three features, a student gains a simpler, faster-to-use rule but loses worst-case predictive fit, since the residual level increases from 126.5214 to 196.8252.